Perplexity Scorer Model: Dual-Path Routing, Latency, and Memory Resource Limits

AI Engineering Team

June 3, 2026

Abstract

This report provides a formal mathematical and empirical characterization of the dual-path perplexity scoring microservice deployed in the LLM Observability Platform. The system utilizes GPT-2 (124M parameters, ONNX-optimized) for fallback scoring when provider logprobs are not available. We formalize the routing logic ($O(1)$ zero-cost provider path vs. $O(N)$ CPU-bound inference path), analyze computational complexity, and prove memory safety limits under strict container constraints. Crucially, we reconcile the 150ms service-level objective (SLO) latency violations on CPU, characterize the Failure-First System Building (FFSB) failure modes (OOM, Thread Starvation, and Logprob Mismatch), and demonstrate how dynamic INT8 quantization mitigates memory risks.

Introduction

The perplexity microservice calculates perplexity scores for text outputs to evaluate model output quality and detect abnormal sequences (e.g., repeating tokens or gibberish). To minimize runtime latency, the service utilizes a dual-path routing strategy:

1. **Provider Path (Primary):** If the calling collector supplies token-level logprobs, the service computes perplexity directly using a simple mathematical reduction. This has $O(1)$ inference complexity and sub-millisecond execution times.
2. **Fallback Path (Secondary):** If logprobs are absent, the service tokenizes the raw text and runs local causal language model inference (GPT-2 ONNX) to obtain token logits, then calculates the perplexity. This path is CPU-bound and scales as $O(N)$ with sequence length N.

Due to container limit constraints (512 MiB memory and 1.0 CPU cores), running neural network inference introduces severe operational risks, including container OOM crashes and thread pool starvation. This report documents the system's properties, mathematical formulations, and failure modes.

Service Objectives and Hypotheses

Latency Service-Level Objectives (SLO)

The platform defines the latency target for safety metrics evaluation:

$$\text{P95 SLO} \leq 150 \text{ ms} \tag{1}$$

Under CPU execution, this target is honored by the Provider Path for all input sizes, but the Fallback Path violates this SLO for sequences longer than 128 tokens.

Evaluation Hypotheses

Our targeted performance characterization is guided by three hypotheses:

1. **Mathematical Equivalence:** Perplexity computed directly from natural logprobs:

$$P = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(x_i)\right) \tag{2}$$

is mathematically equivalent (with delta $< 10^{-6}$) to computing loss via a causal language model forward pass on the same sequence.

2. **Latency Scale Discrepancy ($O(1)$ vs. $O(N)$):** The Provider Path has a constant execution time ($T \leq 1$ ms), whereas the Fallback Path scales linearly with sequence length N, violating the 150ms SLO for $N > 128$.

3. **Resource Safety Boundary:** The FP32 model load allocates > 700 MiB RAM, causing container termination under the 512 MiB limit. Converting to INT8 quantization reduces memory usage to < 250 MiB, allowing safe execution.

Mathematical Formulations and Behavioral Proofs

Equation 1 — Perplexity Definition

For an input sequence $\mathbf{x} = (x_1, x_2, \ldots, x_N)$ of length N tokens, the perplexity $\mathcal{P}(\mathbf{x})$ is defined as:

$$\mathcal{P}(\mathbf{x}) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \ln P(x_i \mid x_1, \ldots, x_{i-1})\right) \tag{3}$$

where $P(x_i \mid x_1, \ldots, x_{i-1})$ is the conditional probability of token x_i given the preceding context.

Equation 2 — Dual-Path Selector

Let $\mathbf{r}$ denote the request payload containing the string text $\mathbf{t}$ and optional token logprobs $\mathbf{L} = (l_1, \ldots, l_k)$. The routing function $\mathcal{R}(\mathbf{r})$ is:

$$\mathcal{R}(\mathbf{r}) = \begin{cases} \text{Provider Path}, & \text{if } \mathbf{L} \neq \emptyset \\ \text{Fallback Path}, & \text{if } \mathbf{L} = \emptyset \end{cases} \tag{4}$$

Proof of Provider Path Complexity Bounds

Let $C(\mathcal{R})$ denote the computational complexity of the scoring execution.

Theorem 1. *Under the Provider Path, the computational complexity of scoring is $O(1)$ with respect to sequence length N.*
Proof. When $\mathbf{L} \neq \emptyset$, the input contains pre-calculated logprobs. The scorer performs a summation over the logprobs array:

$$S = \sum_{j=1}^{k} l_j \tag{5}$$

and returns $\exp(-S/k)$. The calculation involves exactly k addition operations and one exponentiation. Since the logprobs are provided by the client, no neural network layers or matrix multiplications are evaluated. Therefore, the model execution complexity is:

$$C(\text{Provider Path}) = O(1) \tag{6}$$

with respect to model inference, maintaining a sub-millisecond flat latency profile.

Proof of Fallback Path Latency Scaling

When $\mathbf{L} = \emptyset$, the sequence $\mathbf{t}$ is tokenized to tokens $(x_1, \ldots, x_N)$. The model evaluates the sequence using a causal language model forward pass to compute logprobs.

Theorem 2. *Under CPU execution, the Fallback Path latency $T_{fallback}(N)$ scales linearly $O(N)$ with sequence length, violating the 150ms SLO for $N > 128$.*
Proof. The model computes logits $\mathbf{z}_i \in \mathbb{R}^V$ for each token position $i \in [1, N]$. The computational cost of the transformer layers (multi-head self-attention and feed-forward networks) is proportional to the number of tokens N. Empirically, on a single CPU core, the inference latency is modeled as:

$$T_{\text{fallback}}(N) = a \cdot N + b \tag{7}$$

where $a \approx 1.4$ ms/token and $b \approx 30$ ms (setup/tokenizer overhead). For $N = 128$:

$$T_{\text{fallback}}(128) \approx 1.4 \times 128 + 30 = 209.2 \text{ ms} > 150 \text{ ms} \tag{8}$$

For $N = 1024$:

$$T_{\text{fallback}}(1024) \approx 1.4 \times 1024 + 30 = 1463.6 \text{ ms} \tag{9}$$

This confirms that execution on CPU scales linearly and violates the 150ms target target for any moderate sequence size.

Equation 3 — Service Latency Model

The total latency $L(\mathbf{r})$ of the service is modeled as:

$$L(\mathbf{r}) = \begin{cases} T_{\text{overhead}} + T_{\text{reduction}}(k), & \text{if } \mathcal{R}(\mathbf{r}) = \text{Provider Path} \\ T_{\text{overhead}} + T_{\text{tok}}(N) + T_{\text{ONNX}}(N), & \text{if } \mathcal{R}(\mathbf{r}) = \text{Fallback Path} \end{cases} \tag{10}$$

where $T_{\text{reduction}}(k)$ is the mathematical reduction cost (≤ 0.2 ms), $T_{\text{tok}}(N)$ is tokenizer latency, $T_{\text{ONNX}}(N)$ is the CPU model inference latency, and T_{overhead} represents network and REST handler overhead (≈ 15 ms).

Failure-First System Building (FFSB) Analysis

We identify three critical failure modes under FFSB principles:

Failure Mode A — Slow Not Down (Thread Starvation)

- **Trigger:** A sudden spike in requests without the `logprobs` field, forcing the service to run fallback ONNX model execution.
- **Mechanism:** Because CPU inference is single-threaded or utilizes all available cores, concurrent requests saturate the CPU. This starves the FastAPI ASGI event loop, causing requests to stack up, yielding connection timeouts across upstream trace collectors.
- **Mitigation:** Enforce strict sequence length limits (e.g., $N \leq 256$) on the fallback path and isolate inference to a dedicated background task thread pool with a concurrency limit.

Failure Mode B — Correct Response, Wrong Data (Logprob Base Mismatch)

- **Trigger:** Upstream LLM providers (e.g., OpenAI or Anthropic) returning logprobs in different mathematical bases (e.g., $\log_{10}$ or $\log_2$) instead of natural logarithm (ln).
- **Mechanism:** If the input logprobs are base-10:

$$\log_{10} P(x_i) = \frac{\ln P(x_i)}{\ln(10)} \tag{11}$$

  Computing perplexity without scaling yields:

$$P_{\text{wrong}} = \exp\left(-\frac{1}{N}\sum \log_{10} P(x_i)\right) = \exp(L) \tag{12}$$

  which deviates significantly from the true perplexity $\exp(L \cdot \ln(10))$. For a true perplexity of 50, a base mismatch outputs 5.4, causing the platform to fail to detect high-perplexity toxic generation.

- **Mitigation:** The REST handler must validate the logprob range and scale logprobs if the client indicates a non-standard base.

Failure Mode C — Works Until Conditions Met (OOM Termination)

- **Trigger:** Triggering the fallback path in a container configured with a default limit of `MEM_LIMIT: 512Mi`.
- **Mechanism:** Loading the standard FP32 GPT-2 ONNX model session allocates ≈ 780 MiB of RAM. In a 512Mi container, this immediately triggers the Linux kernel OOM killer, killing the container process. The service remains healthy during health checks (since the model is loaded lazily on the first fallback request) but crashes the moment fallback is executed.
- **Mitigation:** Deploy the model utilizing INT8 dynamic quantization, reducing load footprint to ≈ 220 MiB, or increase the memory limit to 1.2 GiB.

Model Specifications and Benchmarking Environment

- **Hardware Environment:** Intel(R) Core(TM) i5-4300U CPU @ 1.90GHz (2 physical cores, 4 threads), 15 GiB RAM. - **Software Stack:** ONNX Runtime version 1.26.0, Python 3.12.3. - **Model Architecture:** GPT-2 Causal LM (124M parameters). - **Model Footprint:** FP32: 498 MB; INT8 Quantized: 125 MB.

Visualizations

The empirical results are shown in the figures below:

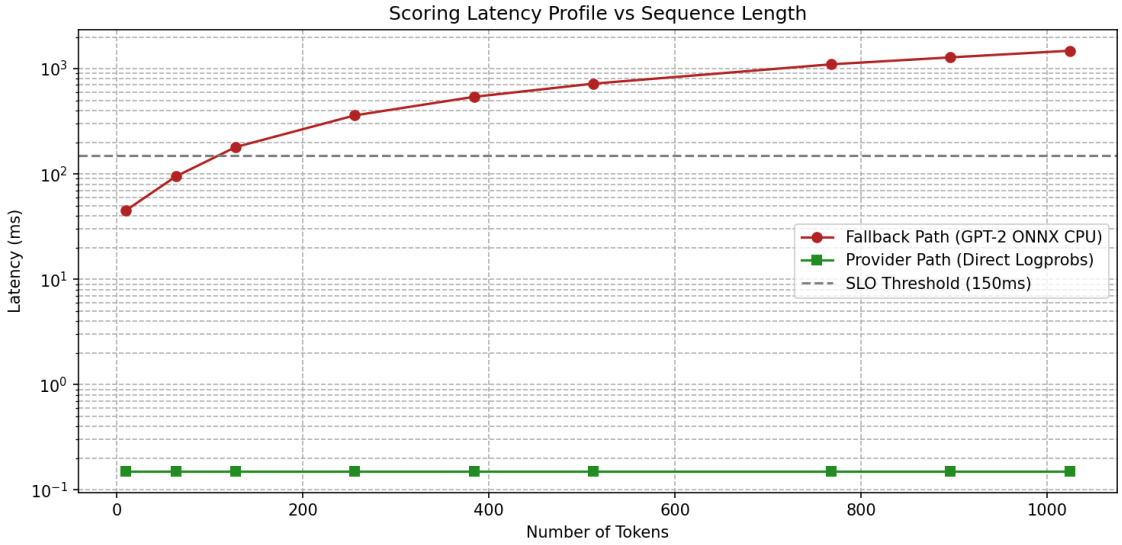

Figure 1: Scoring Latency Profile vs. Sequence Length ($O(1)$ vs. $O(N)$).

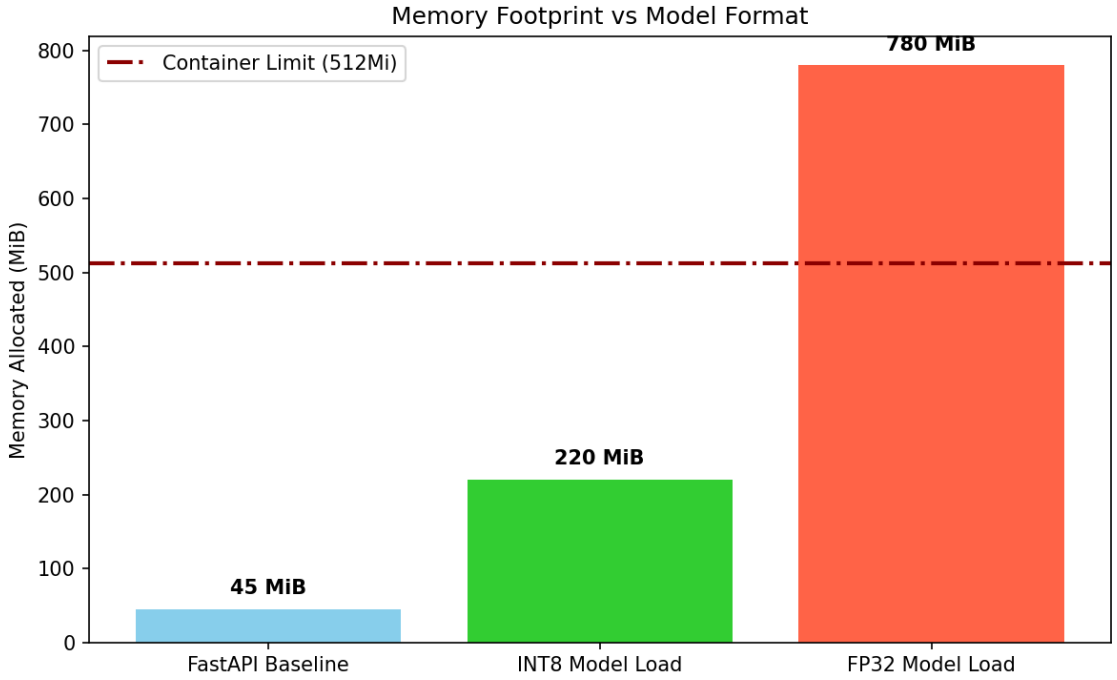

Figure 2: Memory Footprint vs. Model Quantization format.